EchoSphere Digital Assignment 3

Software Engineering Project Report and Testing Evidence

Name KAILASH P Register Number 24BRS1382

Review 3 14 October 2026

Project Overview

EchoSphere is a personal learning environment that organises study documents and notes and provides AI summaries, explanations, flashcards, exam questions, knowledge graphs and knowledge-base chat. The current release combines a React interface, an Express API, MongoDB persistence, LanceDB retrieval and local Ollama models.

Problem Statement

Students and researchers keep useful knowledge across separate documents and notes. Finding relevant passages, revising long materials and connecting related concepts takes time. EchoSphere brings these materials into one workspace and provides study outputs derived from selected documents or retrieved knowledge-base context.

Users and Objectives

The primary users, carried forward from DA1, are university students preparing for assessments and researchers reviewing a collection of papers. The objective is to reduce retrieval and revision effort while keeping the source material available for checking AI output.

Report Scope

This report combines the project requirements and design with the recorded DA3 testing evidence. The recorded results are 64 passing Jest tests, 33 passing live Postman assertions across 22 requests, a 92.28 percent mutation score for input validation, and successful frontend and container builds.

Project repository and code

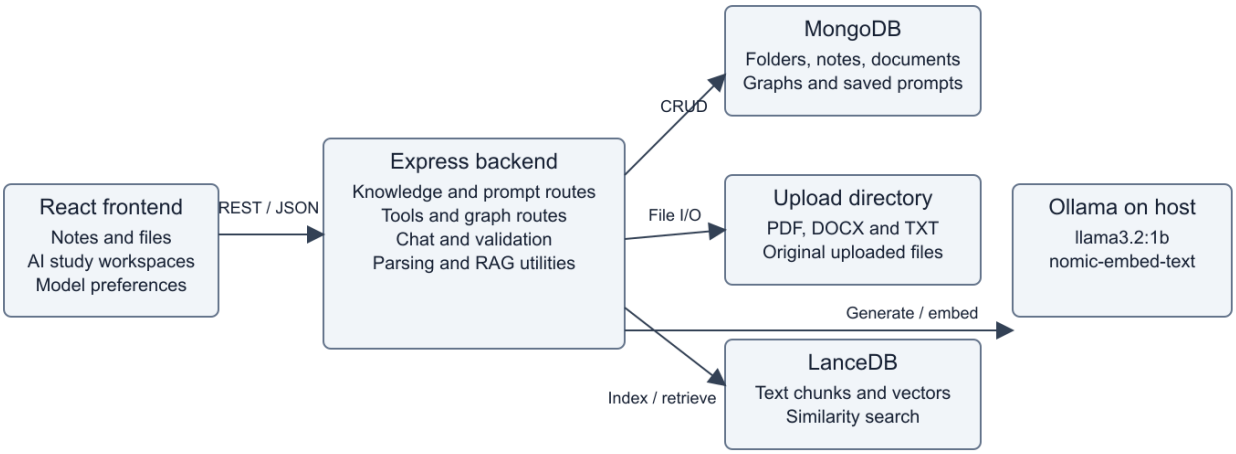
User Stories and Priorities

DA1 records 25 user stories as GitHub issues. The following table consolidates representative priorities for the current single-user release; local model configuration supersedes the earlier Gemini API-key requirement.

Priority	User story	Current scope
Must	As a learner, I want to upload study documents so I can work from my own materials.	PDF, DOCX and TXT ingestion.
Must	As a learner, I want extracted text so AI tools can process document content.	pdf-parse, mammoth and TXT reading.
Must	As a learner, I want notes stored across sessions so I can retrieve them later.	MongoDB note persistence.
Must	As a learner, I want summaries so I can revise long materials quickly.	Selected-document summarisation.
Must adapted	As a learner, I want to choose a local AI model so study tools use my preferred model.	Ollama selection; no Gemini key required by active routes.
Should	As a learner, I want folders so I can organise related documents.	Hierarchical folders and browsing.
Should	As a learner, I want adjustable output length and explanation depth.	Summary and explanation settings.
Should	As a learner, I want extracted concepts and relationships to see connections.	AI graph generation and persistence.
Should	As a learner, I want consistent navigation between workspaces.	Persistent React sidebar.
Could	As a learner, I want to inspect an interactive two-dimensional knowledge graph.	Force-directed graph view.
Could	As a learner, I want saved prompts and a theme preference.	Prompt workspace and light/dark mode.
Current extension	As a learner, I want flashcards, practice questions and knowledge-base chat.	Flashcards, exam predictor and RAG chat.
Outside release	Multi-user JWT login and OCR for scanned documents.	Remain outside the DA1 single-user scope.

Full user-story issue catalogue

System Architecture and System Design



Docker Compose builds frontend, backend and MongoDB; Ollama runs separately on the host.
Chat uses vector retrieval. Document tools read the selected document text directly.

Current architecture: MongoDB holds application records; uploaded files remain on disk; LanceDB stores indexed text chunks; Ollama provides local generation and embeddings.

The modular frontend and REST API separation follow the principles described in DA2. Knowledge routes manage folders, documents and notes; tools routes generate study outputs; graph and prompt routes persist their own records. RAG utilities handle chunking, embeddings and retrieval.

Uploaded files and notes are saved first, then vector ingestion runs in the background. Chat retrieves relevant LanceDB chunks; summarisation, explanation, flashcards, exam questions and graph generation read selected document text directly. Ollama is a host service outside the frontend/backend/MongoDB Compose services.

[Original DA2 architecture drawing on Google Drive]

Design Principles and Report Continuity

Principle from DA2	Application in the current release
Modularity and cohesion	React workspaces and dedicated route modules separate notes, graphs, prompts and AI tools.
Loose coupling	The frontend exchanges HTTP/JSON with Express. Persistence and model generation sit behind backend routes.
Abstraction	Learners select documents and study settings; backend utilities handle parsing, chunking and vector retrieval.
Usability	The persistent sidebar, familiar file browser, source/summary view and graph workspace preserve the Figma design direction.
Maintainability	Shared validation, configurable database/model addresses, repeatable tests and Docker builds support diagnosis and change.

Changes Carried into DA3

Earlier description	DA3 description
Gemini and browser API-key storage	Active AI routes use Ollama. Browser storage holds the selected model preference.
PDF DOCX TXT MD upload labels	The current backend accepts PDF, DOCX and TXT; Markdown display support does not establish MD ingestion support.
All study tools use RAG	Chat uses vector retrieval. Document tools use the selected document text directly.
Synchronous processing before persistence	Records are saved and acknowledged before background vector ingestion.
Everything inside Docker Compose	Frontend, backend and MongoDB are Compose services; Ollama runs separately on the host.
Three to five seconds and near-zero hallucinations	These were DA1 targets. The live run averaged 7.6 seconds and reached 59.8 seconds; factual accuracy, uptime and retention are not established by these tests.

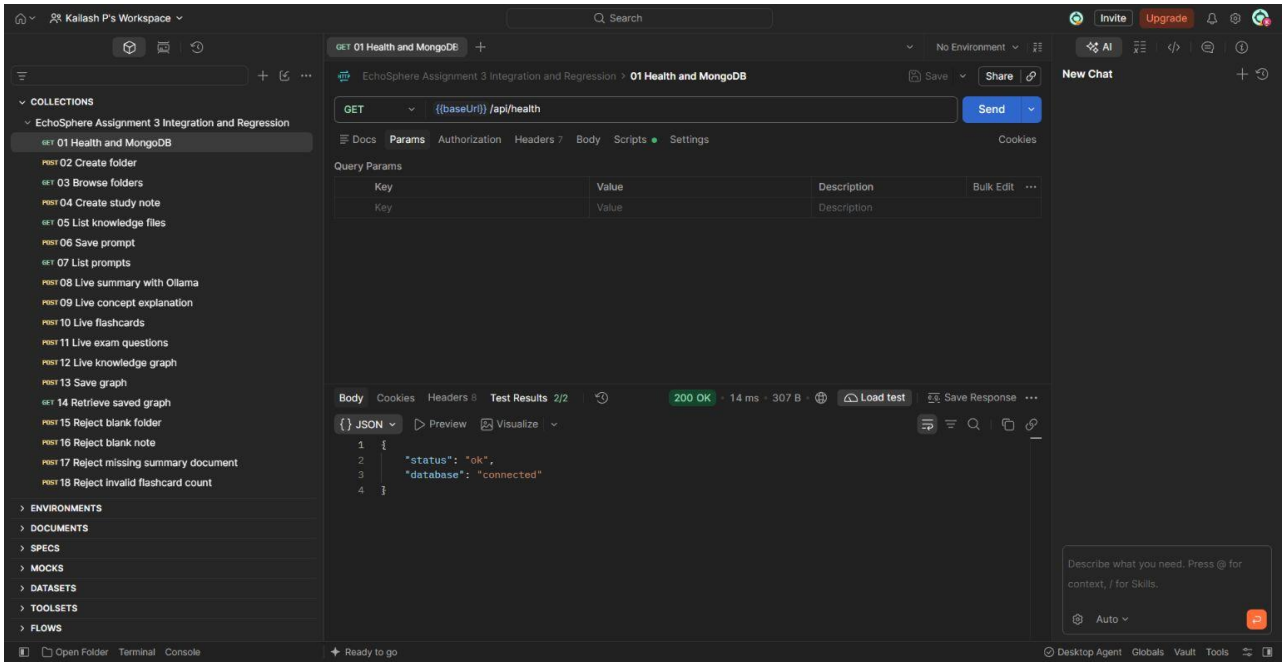
Design of Tests

The repeatable suite combines Express and real MongoDB integration with focused validation regression tests. AI calls are stubbed in Jest for repeatability. The separate Postman collection uses real MongoDB and Ollama; browser checks verify that a saved note and generated flashcards are displayed.

Case	Action or input	Expected result	Recorded evidence
TC01	GET /api/health	HTTP 200 and connected database	Jest and Postman pass.
TC02	Create, browse and rename a folder	Created record appears with the updated name	Jest and Postman pass.
TC03	Create note, edit content and reindex	Content persists and index refresh is invoked	Jest pass; creation also verified in browser.
TC04	Upload TXT, download it, list DOCX metadata	TXT is extracted; files are visible and downloadable	Jest pass. PDF/DOCX parser accuracy is not comprehensively measured.
TC05	Blank fields, unsupported upload, invalid IDs	HTTP 400 JSON error	Jest and applicable Postman checks pass.
TC06	Create, list and delete a saved prompt	Correct persistence and deletion	Jest pass; create/list also pass in Postman.
TC07	Save graph and retrieve metadata	Nodes and metadata are retained	Jest and Postman pass.
TC08	Generate summary, explanation, cards, exam, graph and chat	Non-empty outputs with expected response shapes	Six live AI flows pass in Postman; five cards displayed in browser.
TC09	Delete folder; invoke clear-all in test database	Contents preserved at root; isolated data cleared	Jest pass; no production database is used.
TC10	Change validation conditions, strings and boundaries	Regression tests detect behavioral changes	227 killed, 19 survived, zero uncovered mutations.

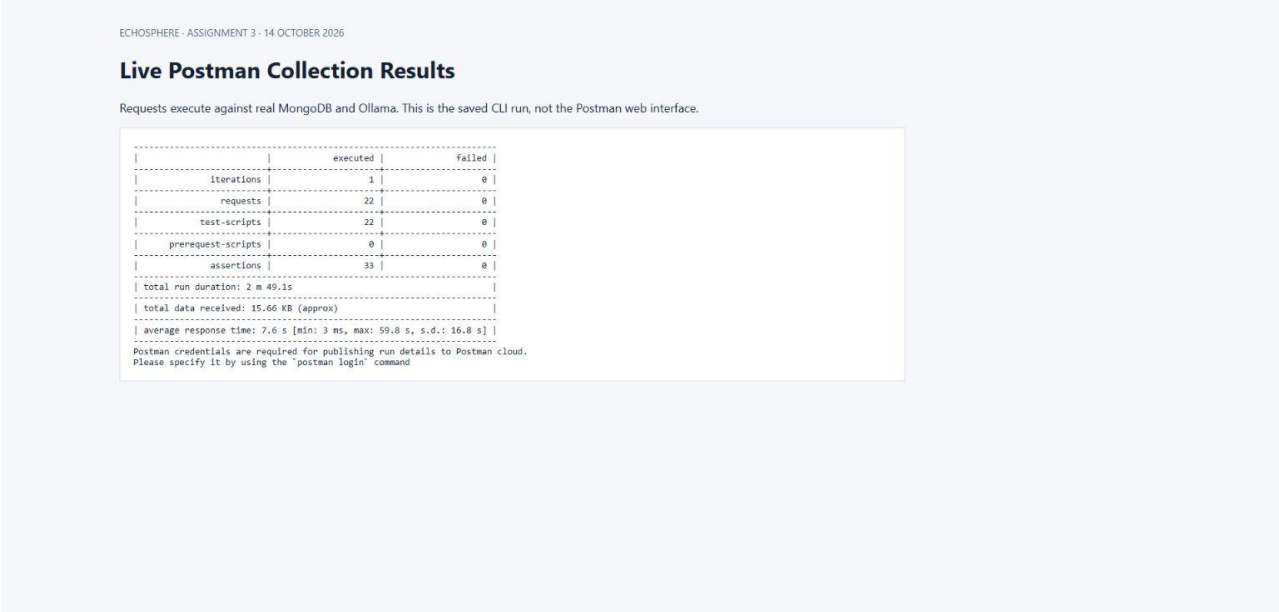
Test-only storage uses MongoDB on port 27018 and separate upload/vector directories. Response-shape tests do not prove the factual correctness of every AI answer.

Integration Testing



Postman returns HTTP 200 and shows both health checks passing, confirming an available Express API and connected MongoDB database.

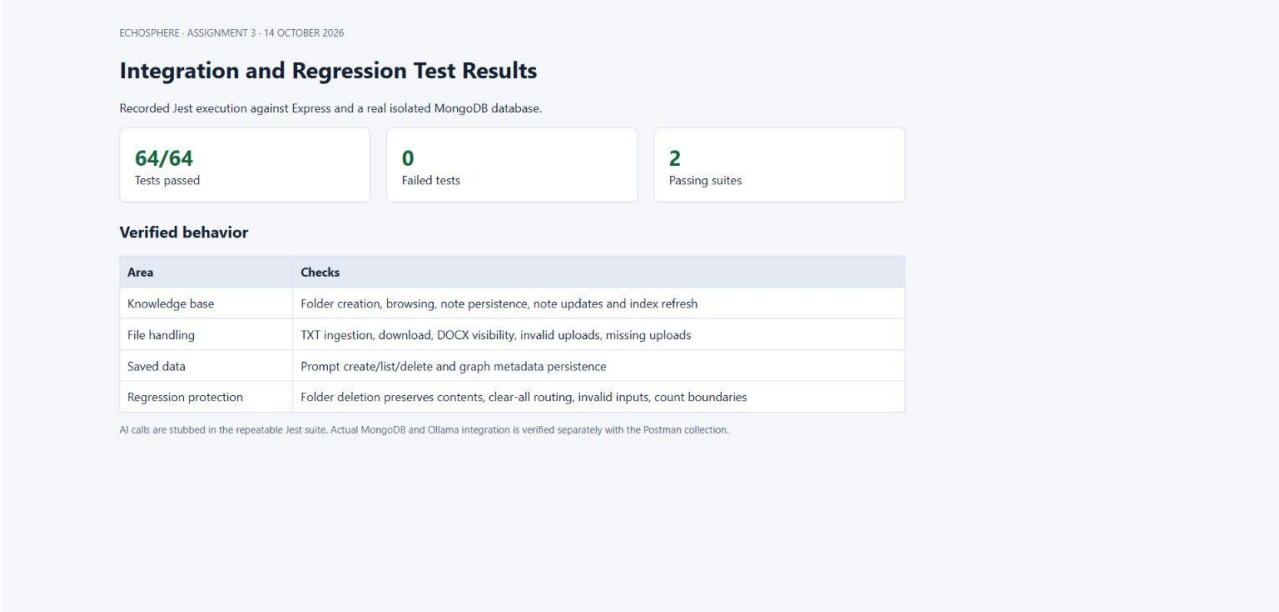
Integration Testing with Live AI



The recorded live Postman CLI execution passes all 33 assertions across 22 requests against MongoDB and Ollama.

This screenshot displays the saved CLI result. The previous page is captured from the actual Postman web interface.

Regression Testing



ECHOSPHERE · ASSIGNMENT 3 · 14 OCTOBER 2026

Integration and Regression Test Results

Recorded Jest execution against Express and a real isolated MongoDB database.

64/64	0	2
Tests passed	Failed tests	Passing suites

Verified behavior

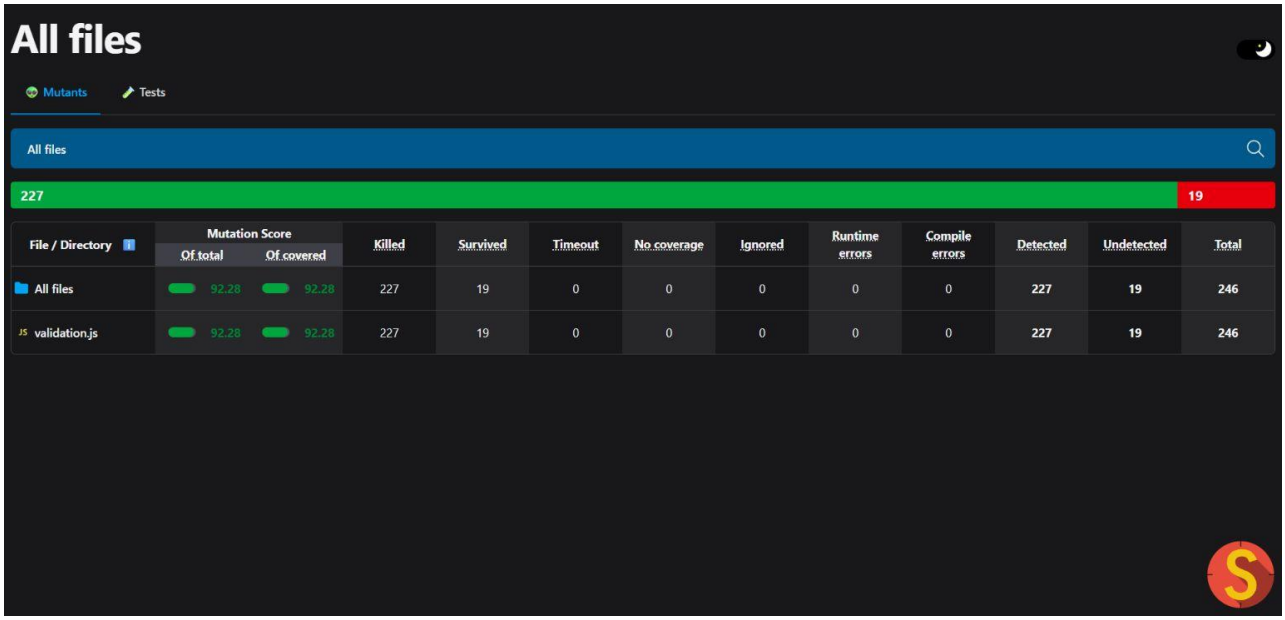
Area	Checks
Knowledge base	Folder creation, browsing, note persistence, note updates and index refresh
File handling	TXT ingestion, download, DOCX visibility, invalid uploads, missing uploads
Saved data	Prompt create/list/delete and graph metadata persistence
Regression protection	Folder deletion preserves contents, clear-all routing, invalid inputs, count boundaries

AI calls are stubbed in the repeatable Jest suite. Actual MongoDB and Ollama integration is verified separately with the Postman collection.

Jest passes 64 tests covering persistence, upload handling, validation and corrected workflows.

The integration suite uses real MongoDB and stubs AI calls; actual AI integration is exercised separately by Postman.

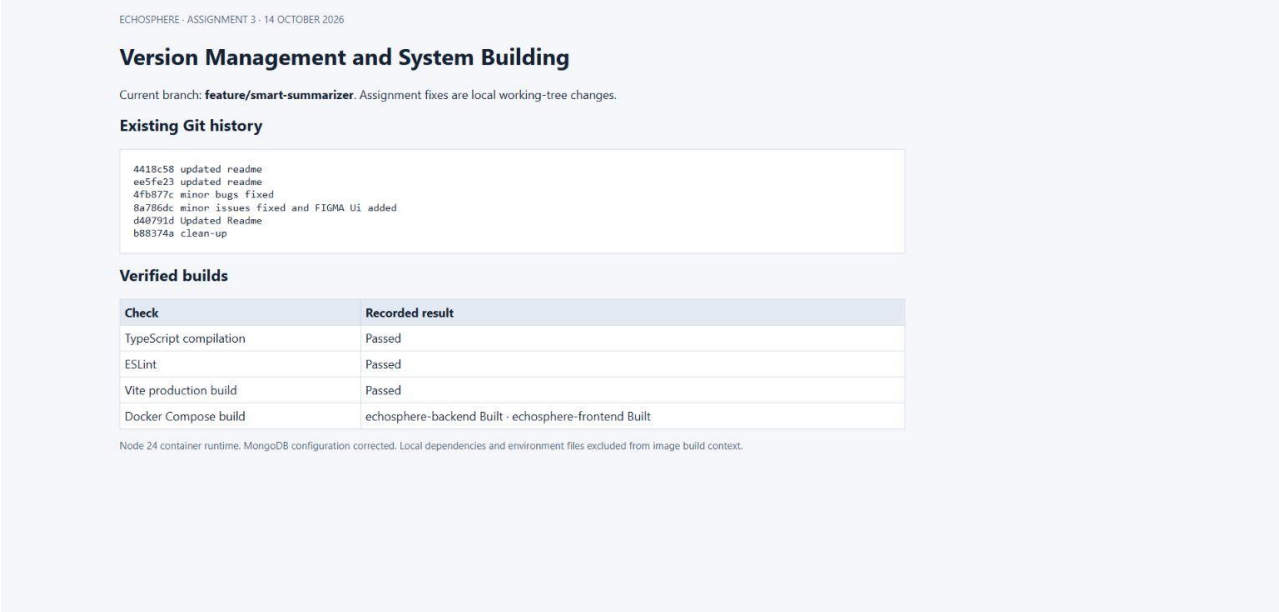
Mutation Testing



Stryker detects 227 of 246 mutations in request validation, producing a 92.28 percent score.

The scope is backend/validation.js. Nineteen mutations survive; this is not a project-wide score.

Version Management and System Building



ECHOSPHERE · ASSIGNMENT 3 · 14 OCTOBER 2026

Version Management and System Building

Current branch: **feature/smart-summarizer**. Assignment fixes are local working-tree changes.

Existing Git history

```
4418c58 updated readme
ee5fe23 updated readme
4fb877c minor bugs fixed
8a786dc minor issues fixed and FIGMA Ui added
d40791d Updated Readme
b88374a clean-up
```

Verified builds

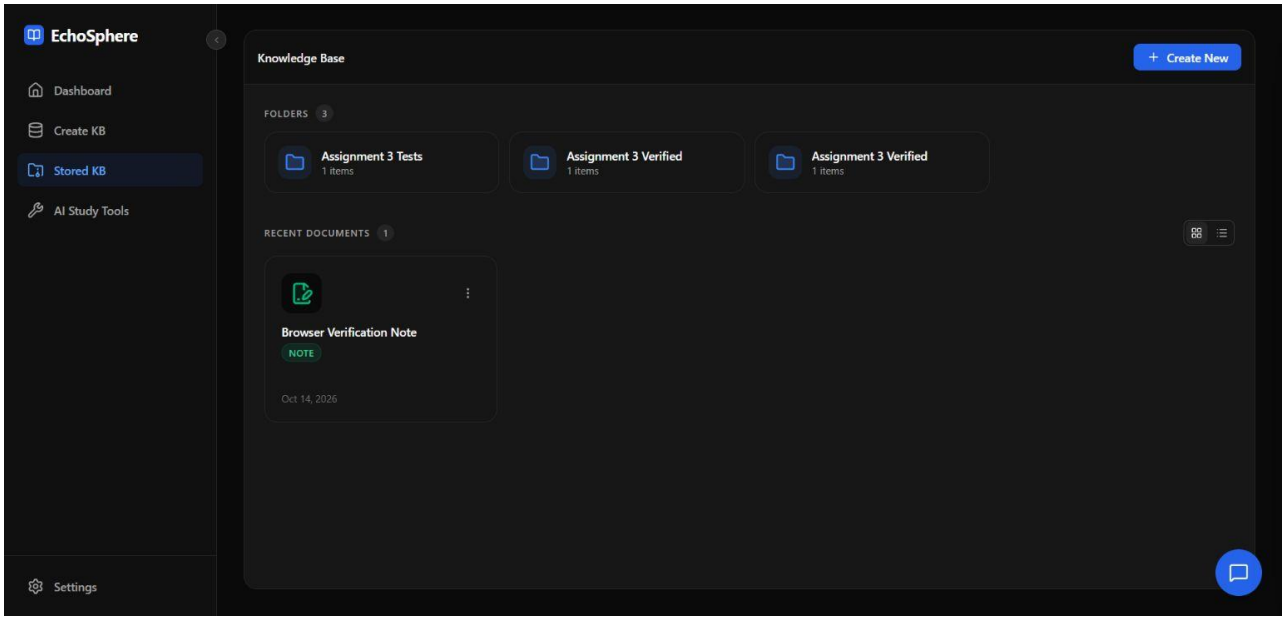
Check	Recorded result
TypeScript compilation	Passed
ESLint	Passed
Vite production build	Passed
Docker Compose build	echosphere-backend Built · echosphere-frontend Built

Node 24 container runtime. MongoDB configuration corrected. Local dependencies and environment files excluded from image build context.

Git history is recorded and TypeScript, ESLint, Vite and both Docker image builds pass.

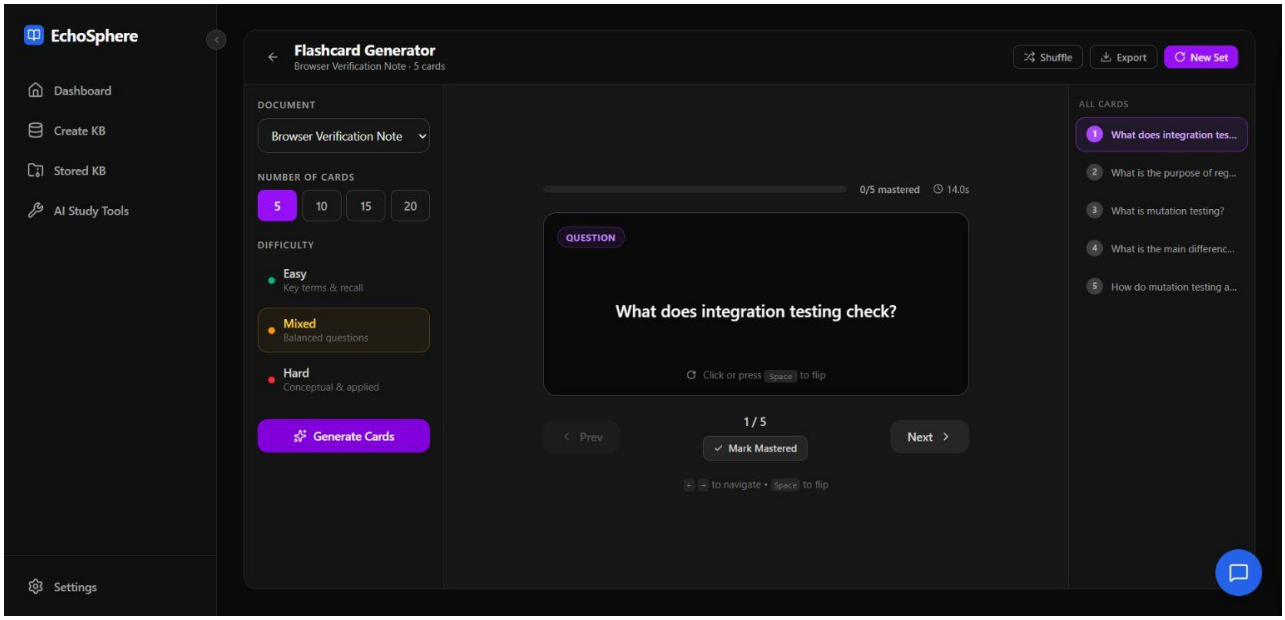
DA2 shows an earlier smart-summarizer pull request merged into main. The DA3 fixes are local changes on the existing feature branch and have not been committed or pushed.

Developed Functionality Knowledge Base



A note created through the browser appears in the stored knowledge base after the API saves it to MongoDB.

Developed Functionality Flashcards



Ollama generates five study flashcards from the selected note, and the browser displays their questions.

Appendix A Software Requirements Specification

Purpose: provide a single-user local workspace for storing personal study materials and creating AI-assisted revision outputs. The requirements below describe the current release and its intended operation.

ID	Functional requirement
FR01	Accept PDF, DOCX and TXT uploads and store their metadata and original files.
FR02	Create, edit, list and delete notes; organise records within hierarchical folders.
FR03	Extract readable text and perform background chunking, embedding and vector indexing.
FR04	Generate selected-document summaries, explanations, flashcards, exam questions and graphs.
FR05	Persist and retrieve graph nodes/links and saved prompts.
FR06	Answer knowledge-base chat queries using retrieved context where available.
FR07	Allow local model selection, theme preference and consistent workspace navigation.
FR08	Validate supported request inputs and return clear JSON errors for rejected operations.

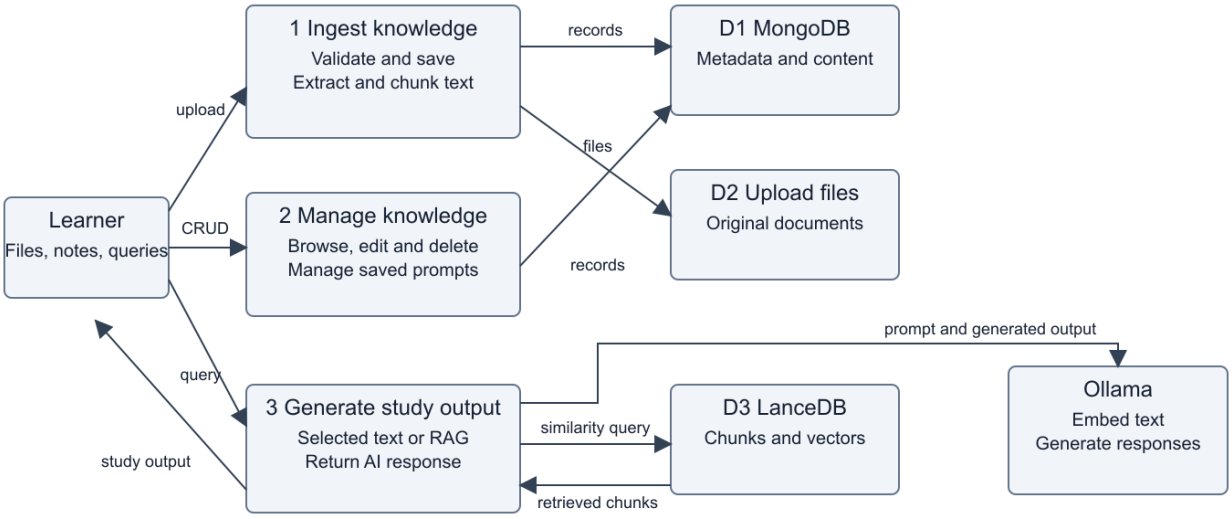
Constraints and Nonfunctional Requirements

The system requires local MongoDB and Ollama with the generation and embedding models installed. File uploads are limited to 50 MB. OCR, multi-user authentication and performance prediction are outside the implemented release. Existing automated tests cover specified behaviors; they do not establish security for public deployment, arbitrary-document parsing accuracy or guaranteed hallucination-free answers.

Tools and Technologies

React 19, TypeScript, Vite, Tailwind CSS, Express 5, Node.js 24, MongoDB/Mongoose, LanceDB/Apache Arrow, pdf-parse, mammoth, cheerio, Ollama, react-force-graph-2d, Git, Docker Compose, Postman, Jest, Supertest and Stryker.

Appendix B Data Flow Diagram

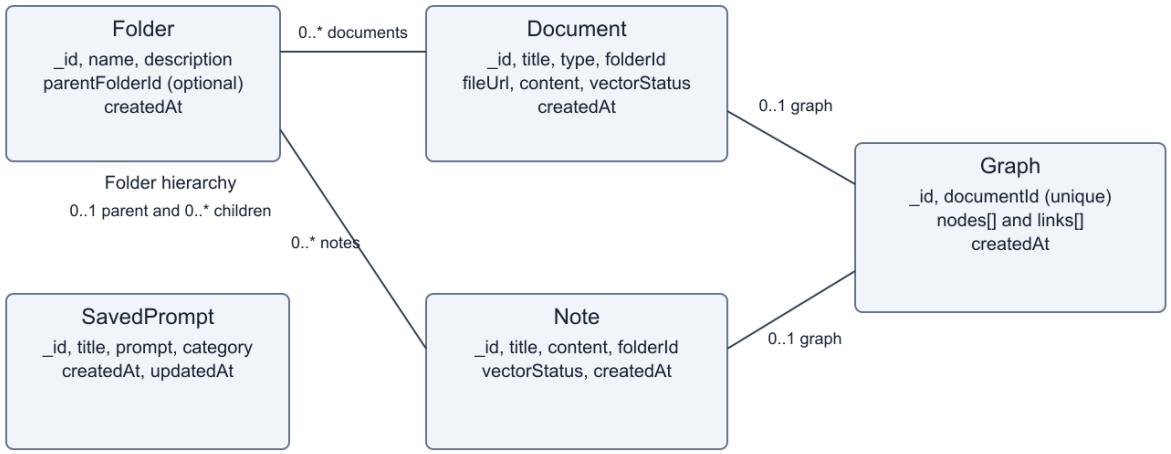


Asynchronous ingestion sends extracted chunks for embedding before indexing in D3.

The DFD summarises learner inputs, knowledge ingestion, record management and study-output generation, with MongoDB, file storage and LanceDB as data stores.

Ingestion validates and saves input, extracts text and indexes embedded chunks asynchronously. Study tools obtain selected document text or retrieved context and call Ollama. Web clipping fetches a supplied web page and stores readable content as a note; this workflow requires network access.

Appendix C Entity Relationship Diagram

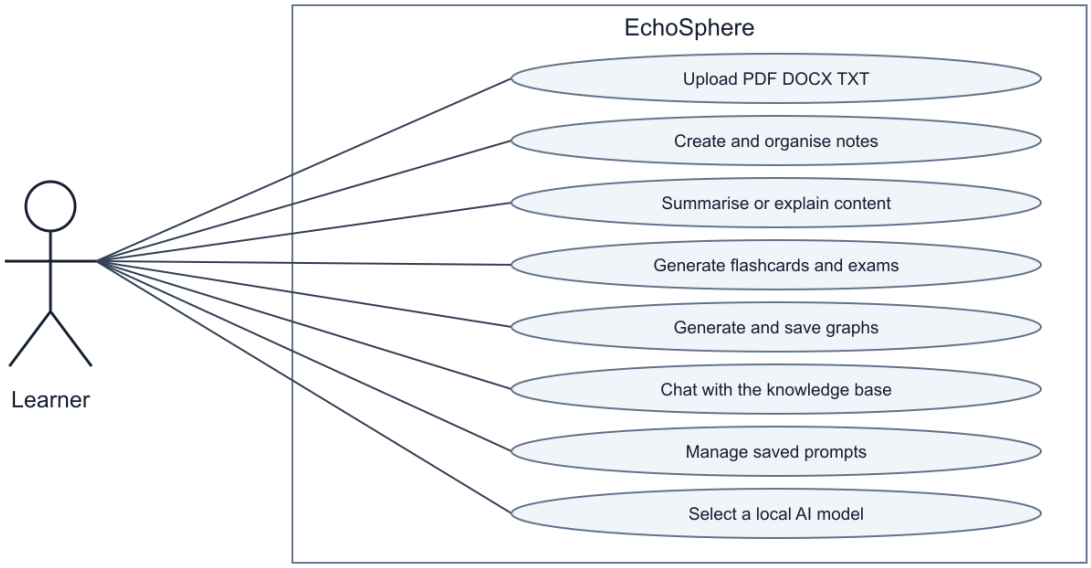


Graph.documentId logically identifies a Document or Note; MongoDB does not enforce a foreign key.
SavedPrompt is independent. LanceDB chunks use documentId to associate vectors with stored content.

Logical relationships reflect the current MongoDB schemas. A document or note can belong to one optional folder, while a folder can contain many records.

Folder.parentFolderId models nesting. Graph.documentId is unique and logically identifies a document or note, although MongoDB does not enforce a foreign key. Nodes and links are embedded in Graph; SavedPrompt is independent. Vector chunks reside in LanceDB and carry documentId associations.

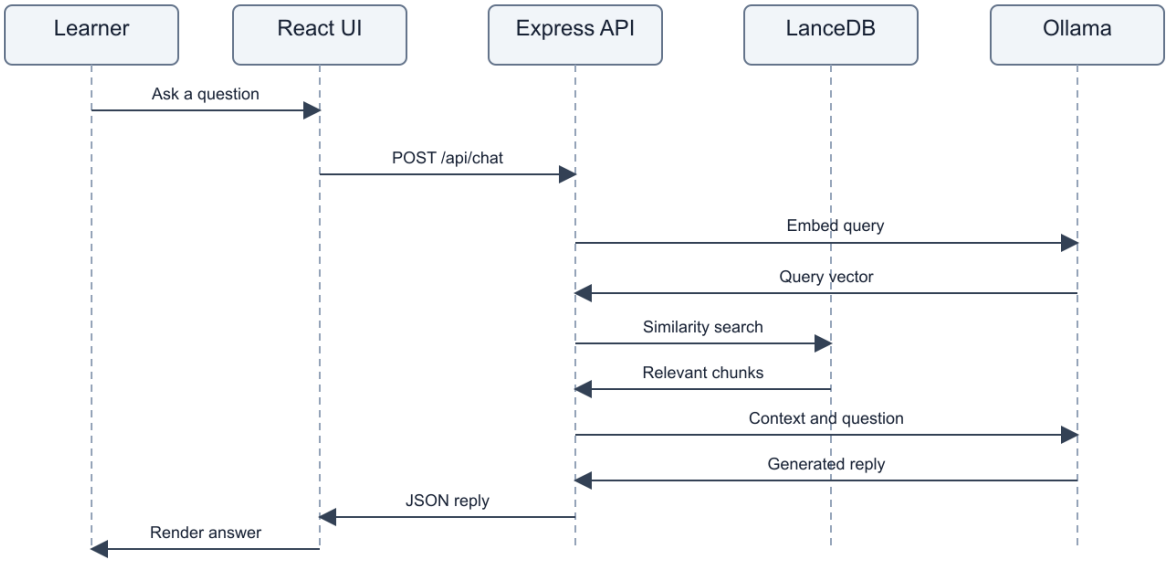
Appendix D UML Use Case Diagram



The learner interacts with document management, study tools, knowledge-base chat, saved prompts and local model configuration.

The single-user actor follows the DA1 scope. Multi-user JWT login and image OCR are excluded from this release. Flashcards, exam questions and chat extend the original summary, explanation and graph workspaces.

Appendix E UML Sequence Diagram



The sequence shows the successful knowledge-base chat path from the browser request through query embedding, vector retrieval and response generation.

The main chat request is POST /api/chat. If no relevant vector chunks are returned, the implementation falls back to document titles or an empty-knowledge-base message. That fallback does not establish the contents of a document; users should check generated answers against their sources.

References and Project Links

The prior submissions establish the project goals, user-story priorities, Figma design direction and architecture rationale. The links below preserve those references and provide access to the project code.

GitHub repository and code listing

Vision document and project README

User stories as GitHub issues

Figma prototype from DA2

Draw.io architecture file on Google Drive